

ChunkyMemo : Un jeu interactif pour explorer les limites de la mémoire de travail*

Eric-Nicolas BRIEAU¹, Omar EL MOUJAHID¹, and Salma ZAIME¹

¹ ENSIM, 1 Rue Aristote, 72000 Le Mans

Abstract. ChunkyMemo est un jeu interactif qui permet de vivre en temps réel les limites de la mémoire de travail. En utilisant le clavier pour reproduire des séquences de flèches, et des capteurs physiologiques (PPG et PZT) pour mesurer la charge cognitive, le projet illustre la loi de Miller (7 ± 2) et l'effet du chunking. Nous expliquons ici la conception, l'implémentation technique et les enseignements tirés de ce projet.

Keywords: Mémoire de travail · BITalino · Charge cognitive · Jeu interactif · Physiologie

* Encadré par Emmanuel BLANCHARD

1 Introduction

ChunkyMemo est né d’une envie simple de rendre visible et concret un concept théorique du cours : la mémoire de travail. Au lieu de lire des articles ou regarder des schémas, nous voulions que le joueur **ressente** lui-même lorsque son cerveau arrive à saturation. Le jeu propose de mémoriser et reproduire des séquences de flèches avec des touches (via le clavier de son ordinateur). Lorsque la séquence devient trop longue, des erreurs arrivent, et les capteurs physiologiques le montrent en temps réel. Ce projet combine jeu vidéo, capteurs BITalino et traitement de signaux pour créer une expérience à la fois ludique et scientifique. La vidéo présentant le projet est trouvable ici [1].

2 Motivation

2.1 Mémoire de travail et loi de Miller

La mémoire de travail est un espace mental temporaire gardant et manipulant des informations. En 1956, George A. Miller a montré que l’on peut généralement retenir sur le court terme sept éléments (plus ou moins deux) [2]. Au-delà, cela devient plus difficile et on commet des erreurs. Le **chunking** (regroupement d’éléments en blocs significatifs) permet de contourner cette limite. Par exemple, dix chiffres isolés sont difficiles à retenir, mais une fois regroupés en paires ou en dates, cela devient plus facile.

ChunkyMemo met cela en pratique avec deux modes de jeux :

1. Mode classique : séquences de flèches de plus en plus longues.
2. Mode chunking : les flèches sont présentées par paires (par exemple “haut-droite” dans un seul bloc).

Le joueur voit clairement la différence de performance entre les deux modes.

2.2 Marqueurs physiologiques de la charge cognitive

Nous mesurons ce qu’il se passe dans le corps lorsque le cerveau est en surcharge. Pour cela, nous utilisons deux capteurs physiologiques :

1. **PPG** (photopléthysmographie), posé sur l’index : pour la fréquence cardiaque et l’amplitude de l’onde de pouls (ou PWA : Pulse Wave Analysis) [3].
2. **PZT** (piézo-électrique) en ceinture : pour le rythme respiratoire et la détection d’apnées courtes.

Ces signaux changent lorsque la charge cognitive augmente : cœur qui s’accélère, respiration qui se modifie, vasoconstriction périphérique, etc. [4]

En complément des marqueurs physiologiques, nous relevons deux indicateurs comportementaux directement issus du jeu :

Le taux d’erreur correspond à la proportion de mauvaises réponses sur l’ensemble des touches saisies à un niveau donné. Ce marqueur reflète la dégradation de la précision mnésique sous charge, cohérente avec les effets documentés

du dépassement de la capacité de la mémoire de travail [2]. Le temps de réaction (TR) est mesuré entre l’affichage de la séquence et la première réponse du joueur. Une augmentation du TR indique une charge de traitement élevée : le sujet hésite davantage avant de s’engager dans une réponse. Ces deux mesures sont combinées aux signaux physiologiques dans un indice composite I_{cog} , défini comme la moyenne des z-scores normalisés par rapport à une baseline individuelle :

$$I_{\text{cog}} = \frac{z_{\text{FC}} + z_{\text{PWA}} + z_{\text{RR}} + z_{\text{TR}}}{4} \quad (1)$$

3 Description technique du système

3.1 Architecture matérielle

Nous utilisons la carte BITalino (r)evolution par Bluetooth, à laquelle sont connectés deux capteurs :

1. PPG sur l’index, branché sur le port A3;
2. PZT en ceinture thoracique, branché port A4.

Les gestes du joueur sont faits sur le clavier et l’acquisition se fait à une fréquence de 100 Hz.

3.2 Architecture logicielle

Nous avons utilisé Python à la fois pour le jeu et pour le traitement de signal afin de limiter les soucis d’interopérabilité. Mais Python étant un langage dynamique, et avec plusieurs façons d’installer des bibliothèques tierces, nous avons besoin d’outils externes fiables pour développer le jeu dans de bonnes conditions.

Outils externes

- uv, un gestionnaire de paquets et d’environnements Python. Il est plus performant et a des options par défaut plus sûres que pip, le gestionnaire de paquets standard plus utilisé. Cela nous a fait gagner beaucoup de temps et permis de traquer les dépendances utilisées, surtout que nous utilisons plusieurs machines et systèmes d’exploitation (Windows et GNU/Linux) pour le développement.
- ruff, qui est un formateur et un linter. Il permet de conserver un code avec un style cohérent et de détecter des erreurs sans avoir à exécuter les fichiers.
- pyright, un vérificateur de types. Il assure un typage cohérent dans le code et facilite sa restructuration.

Fonctionnement

L’application est lancée via `main.py`, qui démarre deux sous-processus Python en parallèle :

1. `game_runner.py` : l'interface Pygame, qui gère les états du jeu (menu, tutoriel, calibration, mode classique, mode chunking) et enregistre les événements du joueur : flèches utilisées, temps de réponse, taux d'erreur, etc.
2. `biosignal_monitor.py` : gère l'acquisition BITalino en temps réel et l'affichage des graphiques physiologiques, avec une sauvegarde des données à chaque session.

Les modules `acquisition.py` (thread BITalino), `signal_processing.py` (filtrages et traitement) et `config.py` (paramètres centralisés) sont partagés entre ces deux processus. La communication entre le jeu et le moniteur physiologique se fait via un fichier JSON (`live_events.json`).

3.3 Traitement des signaux

- PPG (Fréquence cardiaque et PWA) : le signal est filtré avec un Butterworth passe-bande (0,7–4,0 Hz). On détecte les pics pour calculer la fréquence cardiaque (FC) et l'amplitude de l'onde de pouls. Une calibration de 30 secondes au repos sert de référence.
- PZT (Respiration) : filtre passe-bande (0,1–0,8 Hz). On calcule le rythme respiratoire et on détecte les apnées (> 4 secondes).
- Indice composite de charge cognitive : on combine FC, PWA, rythme respiratoire et temps de réaction dans un indice I_{cog} . Lorsqu'il dépasse un seuil, le jeu affiche un message de surcharge.

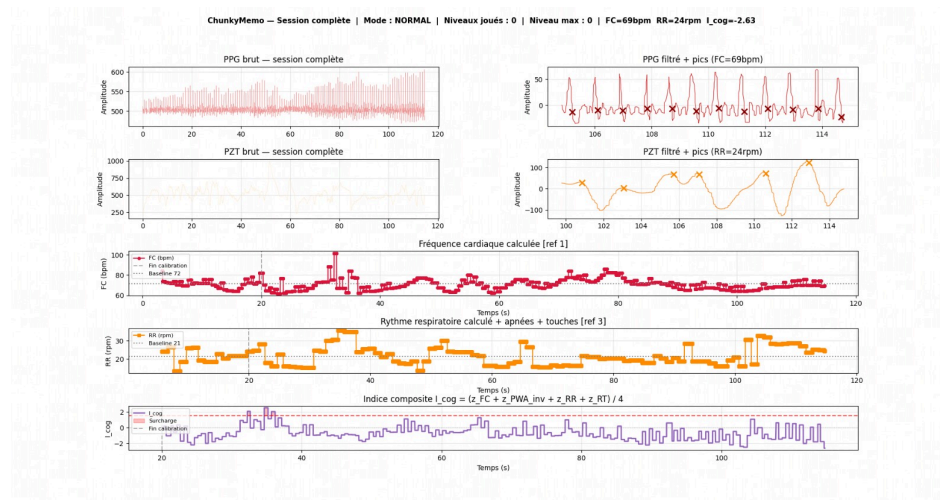


Fig. 1. Capture d'écran de la fenêtre gérée par `biosignal_monitor.py`

4 Développement

4.1 Organisation du projet

Après avoir validé l'idée pendant les séances de travaux pratiques, nous nous sommes répartis les tâches de cette façon :

- Eric-Nicolas s'est principalement occupé du développement des boucles de jeu et de l'interface.
- Omar et Salma ont travaillé ensemble sur la partie acquisition BITalino et traitement des signaux physiologiques. Omar a porté la majeure partie de cette composante technique, notamment à cause des problèmes de connexion Bluetooth rencontrés sur l'ordinateur de Salma. Ils ont donc travaillé en binôme sur la machine d'Omar.
- Salma a également pris en charge la structuration de l'idée du projet, la rédaction du rapport et le montage de la vidéo de présentation.

Nous avons utilisé Github pour versionner l'ensemble du code. Des points quotidiens sur WhatsApp nous permettaient de suivre l'avancée et de nous entraider rapidement.

4.2 Utilisation de l'IA

L'IA a joué un rôle dans la génération et débogage d'une grande partie du code (filtres SciPy, gestion des threads), aide sur les algorithmes de traitement du signal, ainsi que la correction orthographique et l'amélioration du rapport.

Concernant l'interface graphique, l'agent Github Copilot a été utilisé pour accélérer le développement pour des fonctionnalités pouvant être sujettes à erreurs.

Claude (Anthropic) a été utilisé en tant qu'assistant de débogage et de conception : analyse des bugs d'enregistrement des niveaux en mode Chunking, restructuration de la pipeline de traitement de données par niveau, amélioration de la qualité des figures d'analyse, et aide à la rédaction des sections de ce rapport.

Dans les deux cas, le recours à l'IA a été encadré par une relecture critique systématique : chaque suggestion a été comprise, testée et validée avant intégration. L'idéation du projet, la conception de l'architecture logicielle et l'interprétation des résultats sont restées exclusivement le travail de l'équipe.

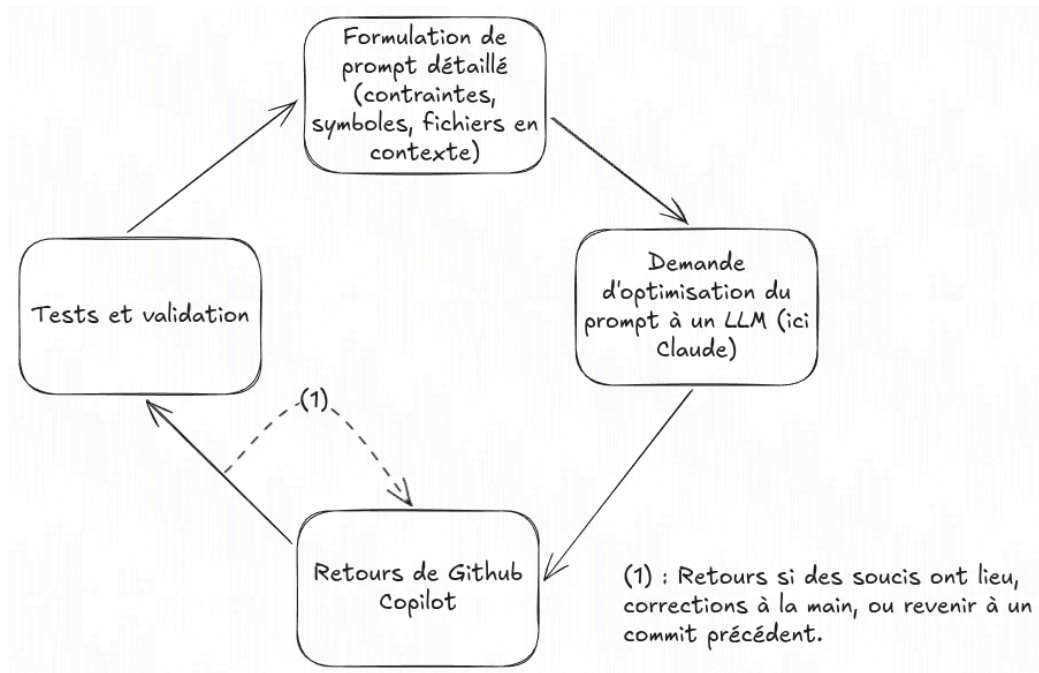


Fig. 2. Cycle de développement assisté par Github Copilot

Difficultés rencontrées

Difficultés matérielles

Une difficulté matérielle récurrente a été la présence de bruit sur les signaux physiologiques, dûs aux mouvements du sujet, à une pose imprécise des capteurs, ou à des défauts d'acquisition de la carte BITalino lors de certains lancements. Ces artefacts se manifestaient par des pics aberrants sur le signal PPG, rendant la détection de fréquence cardiaque instable.

Difficultés logicielles : synchronisation et multithreading

La difficulté logicielle principale a été la synchronisation entre le jeu et l'acquisition physiologique. Ces deux composantes tournent à des fréquences très différentes : le jeu à 60 fps avec une logique événementielle, le BITalino à 100 Hz en continu. Les faire cohabiter dans un même processus s'est révélé problématique.

Notre première approche reposait sur des **threads** Python : un thread principal pour pygame, un thread d'acquisition BITalino, et un thread matplotlib

pour les graphes en temps réel. Cette architecture a posé plusieurs problèmes. Premièrement, pygame et matplotlib ne sont pas thread-safe : appeler leurs fonctions depuis un thread secondaire provoquait des crashes silencieux ou des blocages du rendu. Deuxièmement, le partage de données entre threads nécessitait des verrous (`threading.Lock`) dont l'oubli ou le mauvais placement introduisait des *race conditions* difficiles à reproduire. Troisièmement, la fenêtre matplotlib, utilisant `FuncAnimation` avec le backend `TkAgg`, exige d'être appelée depuis le thread principal, ce qui entraînait en conflit direct avec pygame.

Nous avons donc abandonné l'architecture multi-threadée au profit d'une architecture **multi-processus** : `main.py` lance `game_runner.py` et `biosignal_monitor.py` comme deux sous-processus indépendants via `subprocess.Popen`. Chaque processus possède son propre thread principal et son propre interpréteur Python, éliminant les conflits de rendu. La communication se fait par un fichier JSON intermédiaire (`sessions/live_events.json`) en lecture/écriture partagée : `game_runner.py` y écrit les événements de jeu (niveaux, touches, succès), et `biosignal_monitor.py` le lit pour annoter ses graphes puis l'enrichit avec les métriques physiologiques calculées par niveau à la fin de la session.

Cette solution, bien que plus simple, introduit une latence de lecture fichier de l'ordre de 100 ms, jugée acceptable pour notre usage.

5 Conclusion

ChunkyMemo est une réussite sur plusieurs points. Le jeu est amusant, illustre bien la mémoire de travail et le chunking, et les capteurs apportent une vraie dimension physiologique. Voir son propre rythme cardiaque monter et ses gestes devenir moins précis lorsque la séquence est trop longue est très parlant. Nous avons cependant trouvé quelques points négatifs et améliorations possibles :

- L'indice composite pourrait être affiné avec plus de tests.
- Pas de mode multijoueur ou de statistiques persistantes.

L'absence d'effets sonores et de couleurs variées est volontaire, mais il aurait pu être intéressant de comparer les sessions en prenant ces cas en compte. Cependant cela aurait nécessité un investissement plus long pour le joueur, causant potentiellement une surcharge cognitive par épuisement plutôt que par mémorisation. D'autres configurations auraient posé le même problème, comme l'affichage de flèches avec chacune une durée différente, ou bien l'affichage d'une seule flèche à la fois sur l'écran.

Globalement, ce projet nous a beaucoup appris sur les sciences cognitives, le traitement du signal, le travail en équipe et la gestion d'un projet complet du début à la fin, et nous sommes fiers du résultat.

Bibliography

- [1] S. Z. Eric-Nicolas BRIEAU Omar EL MOUJAHID, “Présentation de Chunkymemo.” [Online]. Available: <https://drive.google.com/file/d/1Fg0q3hkueiuf7ILHbJKJYgvrz35eZABK/view?usp=sharing>
- [2] G. A. Miller, “The Magical Number Seven, Plus or Minus Two: Some Limits on Our Capacity for Processing Information,” *Psychological Review*, vol. 63, no. 2, pp. 81–97, 1956, doi: 10.1037/h0043158.
- [3] J. Allen, “Photoplethysmography and its application in clinical physiological measurement,” *Physiological Measurement*, vol. 28, no. 3, pp. R1–R39, 2007, doi: 10.1088/0967-3334/28/3/R01.
- [4] M. Causse and others, “Task-evoked pulse wave amplitude tracks cognitive load,” *Scientific Reports*, vol. 13, p. 22031, 2023, doi: 10.1038/s41598-023-48917-5.
- [5] M. Grassmann, E. Vlemincx, A. von Leupoldt, J. Mittelstädt, and O. Van den Bergh, “Respiratory changes in response to cognitive load: A systematic review,” *Neural Plasticity*, vol. 2016, p. 8146809, 2016, doi: 10.1155/2016/8146809.
- [6] M. Elgendi, “On the analysis of fingertip photoplethysmogram signals,” *Current Cardiology Reviews*, vol. 8, no. 1, pp. 14–25, 2012, doi: 10.2174/157340312801215782.
- [7] P. H. Charlton and others, “Breathing Rate Estimation from the Electrocardiogram and Photoplethysmogram: A Review,” *IEEE Reviews in Biomedical Engineering*, vol. 11, pp. 2–20, 2018, doi: 10.1109/RBME.2017.2763681.
- [8] P. H. Charlton and others, “Assessing cardiovascular function using an arterial blood pressure waveform,” *Physiological Measurement*, vol. 31, no. 1, pp. R1–R40, 2010.
- [9] A. Welford, *Reaction Times*. Academic Press, 1980.
- [10] W. Hick, “On the rate of gain of information,” *Quarterly Journal of Experimental Psychology*, vol. 4, no. 1, pp. 11–26, 1952.